

Technical Assessment Report: AI-Driven Network Traffic Analysis & Intrusion Detection System

Candidate: Vinodh Kumar

Role Applied: Cybersecurity and Network Security Research Intern

Organization: Megaminds IT Services

Assessment Mode: Round 1 — Take-Home Assignment

GitHub Repository: <https://github.com/vinodhkumar232/megaminds-ids-assessment>

Video Demonstration: [Google Drive Demonstration Link] *(Insert your link here before submission)*

1. Problem Overview

Modern enterprise networks face a constantly evolving threat landscape, encompassing rapid port scanning, volumetric denial-of-service (DoS) attacks, automated brute-force attempts, and stealthy lateral movement. Traditional signature-based Intrusion Detection Systems (IDS), such as classic Snort or Suricata rule sets, are highly effective against known, deterministic exploit signatures. However, they frequently fail when confronted with polymorphic malware, zero-day attack vectors, or distributed campaigns operating beneath static packet-matching thresholds.

Network security teams are increasingly shifting toward behavioral traffic analysis to catch anomalies that evade signature-based systems. Implementing Machine Learning (ML) introduces its own operational challenges: extreme class imbalance (benign traffic dwarfs malicious traffic), severe alert fatigue (flagging individual packets overwhelms Security Operations Center analysts), and the "black-box" nature of neural networks, which complicates forensic triage.

This project addresses these challenges by delivering a working prototype of a **Hybrid Network Flow Intrusion Detection Pipeline**, designed to identify both zero-day anomalies and known attack signatures while dramatically reducing false-positive alert fatigue.

2. Objectives

The primary objective of this assignment is to design, develop, and evaluate a working prototype capable of ingesting network traffic and identifying intrusions. Specific objectives

include:

1. **Data Ingestion:** Process raw network flow data in sequential batches to simulate a live network environment.
2. **Feature Extraction:** Extract and standardize mathematically significant statistical flow features (e.g., timing, packet ratios, TCP flags) without relying on Deep Packet Inspection (DPI), preserving privacy and throughput.
3. **Hybrid Detection Engine:** Combine unsupervised anomaly detection (for zero-day baseline deviations) with supervised classification (for specific attack attribution).
4. **Operational Explainability:** Present output in a manner immediately useful to an analyst, including probabilistic confidence scores and aggregated feature importance for flagged campaigns.

3. Traffic Data & Dataset Description

To train and evaluate the system, the **CIC-IDS-2017** benchmark dataset (provided by the Canadian Institute for Cybersecurity) was selected.

Justification: Older datasets (like KDD99 or NSL-KDD) are obsolete; they fail to represent modern network topologies, lack contemporary background traffic profiles (e.g., HTTPS/TLS), and do not include modern attack vectors. CIC-IDS-2017 provides five days of realistic, multi-vector network traffic.

The dataset includes the mandatory attack profiles required for this assessment:

- **Normal Traffic:** Realistic B-Profile background traffic (HTTP, HTTPS, FTP, SSH).
- **Port Scanning:** Nmap reconnaissance traffic.
- **Denial-of-Service (DoS/DDoS):** Volumetric attacks including LOIC, Hulk, and GoldenEye.
- **Additional Attack (Brute Force):** FTP-Patator and SSH-Patator brute-force login attempts.
- **Ambiguous/Borderline (Infiltration):** Subtle, low-and-slow lateral movement that closely mimics benign traffic.

Data Preprocessing (Hygiene): Raw PCAP-derived flow records often contain parsing anomalies. A custom preprocessing script (`src/merge_datasets.py`) was engineered to strip corrupted headers, replace floating-point infinities (resulting from zero-duration flow divisions), drop null entries, and map the 15 categorical string labels into numeric targets using a `LabelEncoder`.

4. System Architecture

The detection pipeline utilizes a **Hybrid Machine Learning Architecture** featuring two complementary models and a Stateful Sequential Ingestion Engine.

1. **Ingestion Buffer:** A streaming sequential ingestion engine uses a sliding deque memory buffer (length = 30 flows). Rather than predicting on isolated, static rows, it groups traffic chronologically to evaluate *campaign density*.

2. **Unsupervised Behavioral Baseline (Isolation Forest):** The first ML layer. It was trained exclusively on a 200,000-flow sample of *pure benign traffic*. It acts as a statistical low-pass filter, identifying zero-day anomalies and behavioral deviations without needing to know specific attack signatures.
3. **Supervised Signature Classifier (Random Forest):** The second ML layer. Trained on 1.97 million flows across all 15 classes. It analyzes the flow to classify the exact nature of the attack (e.g., DDoS, PortScan) and outputs a probabilistic confidence score.
4. **Campaign Threshold Evaluator:** A logic gate that suppresses alerts unless anomalous flow density exceeds a 30% operational threshold within the monitoring window.

5. Feature Extraction Methodology

Feeding all 80+ raw statistical counters from the dataset into the model introduces severe computational overhead, multicollinearity, and latency penalties. Instead, the architecture extracts a curated **14-dimensional feature vector** focusing on volumetric symmetry, temporal spacing, and L4 signaling.

- **Destination Port:** Identifies targeted services (Port 80/443 for DDoS, Port 21 for FTP).
- **Flow Duration:** Distinguishes rapid port probes (microseconds) from long-lived sessions.
- **Total Fwd/Bwd Packets & Lengths:** Measures client-to-server volume and server responsiveness. Asymmetrical zero-response flows strongly indicate spoofing or scanning.
- **Flow Bytes/s & Packets/s:** Throughput metrics that immediately identify bandwidth-saturating DoS events.
- **Packet Length Mean / Avg:** Evaluates flow symmetry and protocol conformance.
- **TCP Flags (SYN, ACK, PSH, RST):** Crucial for identifying L4 abuse. High SYN counts without ACKs denote SYN floods; high PSH density correlates to interactive brute-force attempts; high RST density indicates aggressive port scanning.

6. Detection Methodology

- **Hybrid Logic:** By passing traffic through both an Isolation Forest and a Random Forest, the system mimics a mature SOC pipeline. If an attack is a known signature, the Random Forest identifies it. If it is a novel zero-day attack, the Random Forest might fail, but the Isolation Forest will successfully flag it as an ANOMALY DEVIATION against the benign baseline.
- **Confidence Scoring:** The Random Forest uses `predict_proba()` to generate confidence percentages. If confidence falls below 85.0%, the severity is dynamically downgraded to UNCERTAIN to prompt manual analyst review.
- **Operational Thresholding (30%):** A critical alert is only triggered if 30% of the flows in the rolling batch are flagged. Triggering on a single anomalous flow (e.g., a misconfigured benign client retry) generates massive false alarm rates. The 30% threshold suppresses transient noise while perfectly capturing sustained volumetric campaigns.

7. Technology Selection

- **Language:** Python 3.10+ (Industry standard for cybersecurity ML, ensures rapid iteration).
- **Framework:** scikit-learn. Chosen over Deep Learning frameworks (TensorFlow/PyTorch) because Random Forests and Isolation Forests provide highly accurate classification on tabular network metadata without requiring expensive GPU acceleration, making the prototype lightweight and portable.
- **Data Processing:** pandas and numpy. Essential for chunking the massive 2.8 million-record dataset in memory and enabling real-time matrix vectorization during inference.

8. Testing and Evaluation

The Random Forest signature classifier was evaluated on an unseen holdout test set comprising 848,423 flows (a 30% stratified split of the master dataset).

Global Metrics:

- **Overall Accuracy:** 98.41%
- **Weighted Average F1-Score:** 0.9895
- **Macro Average F1-Score:** 0.7508

Honest Analysis of Minority Class Performance:

A critical finding is the divergence between the Weighted F1 (0.9895) and the Macro F1 (0.7508). Dominant volumetric classes (DDoS, PortScan, Benign) exhibit near-perfect detection rates (≥ 0.98). However, rare minority classes perform poorly. For example, Web Attacks (XSS, SQLi) frequently occur over standard HTTP ports with statistical metadata identical to normal user browsing. Because our netflow features do not inspect Layer 7 payload strings (DPI), flow-based statistical models struggle to isolate SQLi/XSS.

9. Results (Mandatory Demonstration Scenarios)

A blind inference CLI (src/run_scenarios.py) was engineered to stream raw PCAP flow data where the actual ground-truth labels were hidden from the models, outputting the Baseline Deviation, the ML Prediction, and the Hidden Ground Truth side-by-side.

- **Scenario 1 — Normal Traffic Baseline:** Evaluated pure benign traffic. The Isolation Forest confirmed 100% adherence to the normal baseline. No false alarms were generated, proving the stability of the model.
- **Scenario 2 — Reconnaissance / PortScan:** The models correctly flagged Nmap scanning activity. However, because the scan volume was low (representing < 30% of the active window), the Campaign Evaluator suppressed the critical alert. This successfully demonstrated the alert-fatigue reduction mechanism.
- **Scenario 3 — Denial of Service (DDoS):** Streamed high-velocity DDoS traffic. The

Isolation Forest instantly flagged severe deviations, and the Random Forest classified them with 100.0% confidence. The 30% threshold was breached instantly, triggering a CRITICAL ALERT and outputting the exact features driving the anomaly (Flow Duration, Flow Packets/s).

- **Scenario 4 — Additional Attack (Brute Force):** Streamed FTP-Patator traffic. The engine successfully isolated the short-duration, high-packet-rate anomalies associated with brute-forcing, triggering a high-severity alert and displaying the specific targeting of Destination Port 21.
- **Scenario 5 — Ambiguous/Missed Case (Infiltration):** Streamed stealthy lateral movement (Infiltration) traffic. The model caught the anomaly, but with low confidence (UNCERTAIN, 60.0%). Furthermore, because infiltration is notoriously stealthy and low-volume, it did not breach the 30% threshold. This honestly reveals a limitation: volumetric thresholding is excellent for noisy attacks but struggles with highly stealthy, low-footprint lateral movement, necessitating endpoint log correlation.

10. Network and Security Considerations

1. **Inline Latency vs. Out-of-Band:** Computing 14 statistical features across high-throughput enterprise links (10–100 Gbps) introduces buffer contention. This ML engine should be deployed asynchronously via network tap or SPAN port mirror rather than as an inline blocking IPS.
2. **Encrypted Traffic:** As TLS 1.3 deprecates deep packet inspection, this prototype's reliance entirely on L4 flow metadata (durations, packet symmetries) proves that statistical analysis remains viable even when payloads are encrypted.
3. **Adversarial Evasion:** Attackers can theoretically evade this flow classifier by introducing artificial flow jitter, randomized inter-arrival delays, or padded packets to force their statistical footprint back into the benign baseline.

11. Limitations

1. **Local vs. Global Explainability:** During development, a SHAP TreeExplainer was engineered for exact per-flow local explainability. However, strict endpoint security policies (Windows Defender Application Control) blocked the required Numba DLL injections. To ensure the prototype remained runnable, an AV-safe fallback was deployed that aggregates the top global feature weights for the specific malicious batch.
2. **Temporal Partitioning Leakage:** The evaluation utilizes a random 70/30 stratified split. In network traffic, splitting randomly across the same session can cause data leakage (the model memorizes the session rather than the behavior).

12. Future Improvements

1. **Temporal Cross-Validation:** To resolve data leakage, future iterations will adopt strict temporal partitioning (e.g., training on Monday–Wednesday captures, testing entirely on Thursday–Friday unseen captures) to accurately simulate enterprise deployment.

2. **SIEM Integration:** The current stdout CLI alerts should be formatted into standard JSON/Syslog outputs to allow seamless ingestion into Security Information and Event Management (SIEM) platforms like Splunk or Elastic Security.
3. **Deep Learning for Sequence Modeling:** While the Random Forest is highly efficient, implementing Long Short-Term Memory (LSTM) recurrent neural networks could better analyze the sequential, time-series nature of stealthy attacks like Infiltration.